

Week 2: Profile Page, From Mock Data to the Real Database

Saksham Boran
Roll No: 1024030856

8 September 2026

1 Context

Google Sign-In (college-domain-restricted, via Auth.js v5 with database sessions) and the onboarding flow landed early this week, which meant a signed-in, onboarded user with real `Profile` and `EvidenceRecord` rows in the database finally existed. That unblocked the task queued up from Week 1: a profile page for that user to view their own data.

2 Frontend: Profile Page, First Pass

Built `app/profile/page.tsx` against mock data first, deliberately shaped to mirror the two onboarding-path variants already defined in the schema (Path A: resume/GitHub evidence; Path B: self-reported comfort level, interests, availability), so the layout would be validated against the real data shapes before wiring up any queries. Also added a “View profile” link next to “Sign out” on the signed-in home card, as the only entry point to the new page.

3 Backend: From Mock Data to a Real Query

Once the shape held up, relocated the page into the (`protected`) route group – which gates on completed onboarding at the layout level, using the session from the Google sign-in flow built earlier in the week – and replaced the mock fixtures with a real query:

- Profile fields read from `prisma.profile`.
- Identity (name, email, avatar) read from the session object rather than duplicated into `Profile`.
- Resume/GitHub connection status derived from the user’s `EvidenceRecord` rows (presence/absence and `source` field), rather than a separate boolean flag that could drift out of sync with the actual evidence.

The route group’s layout already redirects a not-yet-onboarded user away, so the page itself only needed a direct-navigation guard for someone hitting the URL manually mid-onboarding – the common path was already covered upstream.

4 Debugging: Broken Avatars With No Fallback

Error encountered: the profile header rendered a plain `<img>` whenever `Profile.image` was non-null. This showed a broken-image icon instead of a sensible fallback, because Google’s avatar URLs (`lh3.googleusercontent.com`) both rate-limit by IP and expire.

Key observation: the existing initials-fallback logic only covered the *null* case (no image URL at all) – a URL that resolves at render time but fails to *load* in the browser is a different failure mode that needs handling on the client, not the server.

Solution: extracted a client component (`Avatar.tsx`) that swaps to initials on the image's `onError` event, plus a mount-time check for failures that happen before hydration attaches that handler (the image is server-rendered, so the browser can fail to load it before any client JS has run):

```
function Avatar({ src, initials }) {
  const [failed, setFailed] = useState(false);
  useEffect(() => {
    // catches a failure that happened before hydration
    if (imgRef.current && !imgRef.current.complete) setFailed(true);
  }, []);
  if (failed || !src) return <InitialsFallback text={initials} />;
  return <img ref={imgRef} src={src} onError={() => setFailed(true)} />;
}
```

5 Verification

No automated test framework exists yet, so verification was manual: a live walkthrough against the real Neon database and a real Google account confirmed the profile page correctly renders both onboarding-path variants, resume/GitHub status reflects the actual `EvidenceRecord` rows (not a stale flag), and a simulated broken avatar URL falls back to initials instead of a broken-image icon.

6 Outcome

Shipped the profile page as three small, separately reviewed commits: mock-data version, database-backed version, then the avatar fix – each landing cleanly rather than as one large first-pass diff.

7 Follow-ups

- At this point the only evidence source was an unverified GitHub URL a user typed in during onboarding – the profile page displayed it as-is. Turning that into a real, verified GitHub connection was the natural next task, picked up in Week 3.
- No automated tests cover the profile page yet; correctness still relies on manual walkthroughs.